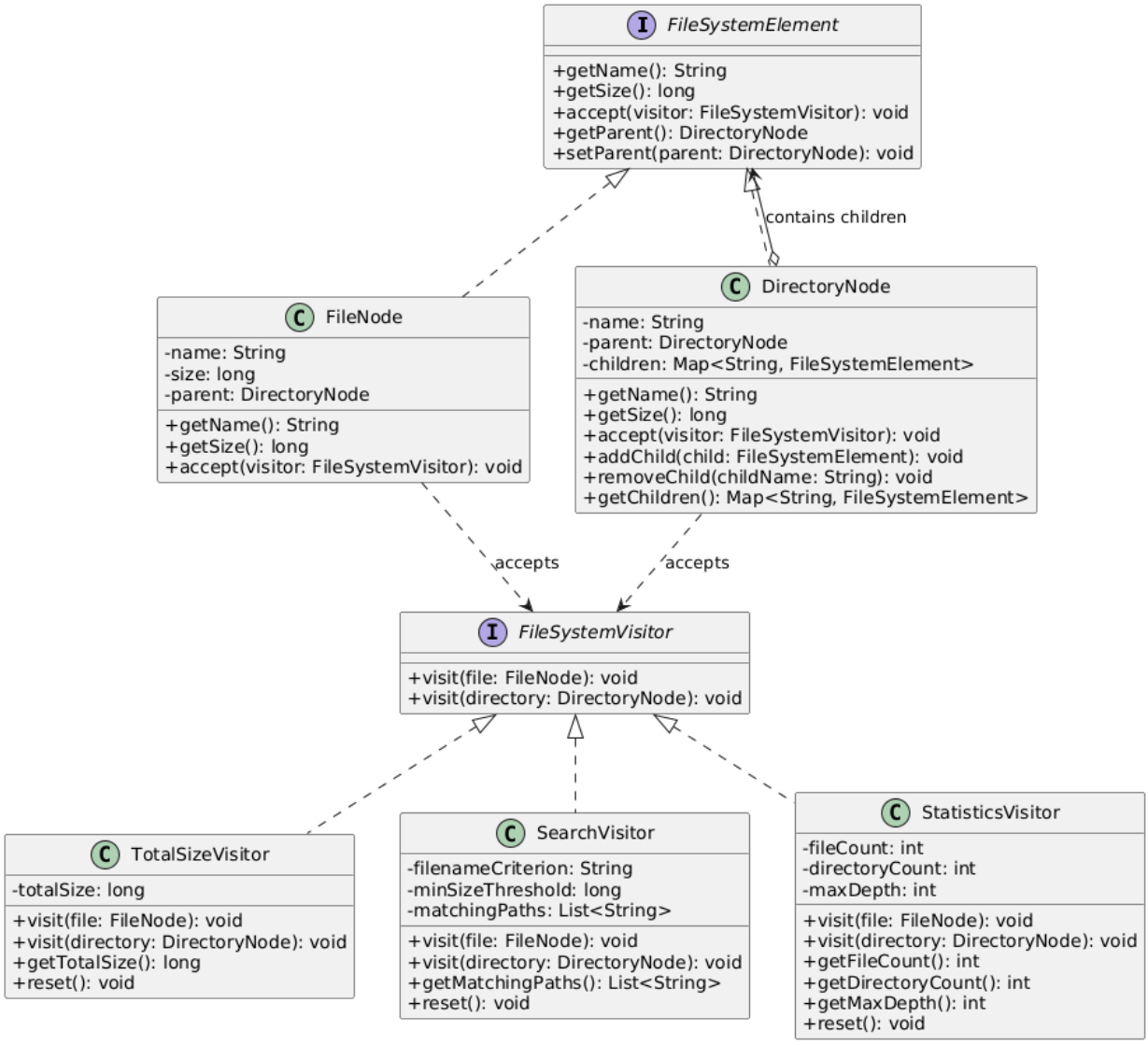
Sanzhar Abayev SE-2538

Project Report: Advanced File System Using the Visitor Pattern

1. Purpose and Design Choices

- **Purpose:** The primary objective is to implement an in-memory hierarchical file system that completely decouples operations from the underlying element nodes (FileNode and DirectoryNode) by leveraging the **Visitor behavioral pattern**.
- **Design Choices:**
  - **Double Dispatch (R2):** Concrete elements implement accept(Visitor visitor), which directly calls visitor.visit(this). This design completely avoids runtime type discrimination or instanceof checks.
  - **Deterministic Traversal (R4, R7):** DirectoryNode maintains its children using a TreeMap<String, FileSystemElement>, ensuring strict lexicographical ordering and predictable root-relative path generation (/ separator).
  - **Tree Integrity Guardrails (R6):** Defensive checks are enforced in addChild() to proactively reject negative file sizes, duplicate sibling names, multi-parent ownership, and cyclic directory references.

2. UML Diagram



3. Five Clean Code Justifications

1. **Meaningful Naming (TotalSizeVisitor)**: Classes and methods are explicitly named after their domain responsibilities (e.g., getTotalSize(), getMatchingPaths()), removing the need for redundant comments.

2. **Single Responsibility Principle (SRP)**: Each visitor encapsulates precisely one algorithm (size summation, file searching, structural metric collection), whereas element classes handle structural state exclusively.

3. **Small Methods**: All visitor visit methods and integrity check routines are kept concise and focused (<15 lines), keeping cyclomatic complexity low.

4. **Encapsulation (DirectoryNode)**: Internal child maps are exposed exclusively via read-only wrappers (Collections.unmodifiableMap) to protect internal directory invariants.

5. **Elimination of Type-Checking Chains**: Branching logic based on concrete types is entirely replaced by polymorphic double dispatch across the visitor contract.

4. Sample Program Outputs

Executing the demonstration application yields the following deterministic output:

```
=== Advanced File System Visitor Pattern Demonstration ===
File system structure initialized successfully.

[Visitor 1 - TotalSizeVisitor]
 -> Total storage size across subtree: 4864 bytes

[Visitor 2 - SearchVisitor]
 -> Matching files (*.java with size >= 500 bytes):
    * /source/Main.java

[Visitor 3 - StatisticsVisitor]
 -> Total File Count: 5
 -> Total Directory Count (including root): 3
 -> Maximum Tree Depth (root = 0): 1

=== Demonstration Completed Successfully ===
```

5. Test Summary

- **Test Class Name:** com.filesystem.FileSystemTest
- **Test Results:** All tests executed successfully via mvn clean test (8/8 verification targets passed).
  - t1_bothElementTypesReachCorrectVisitMethods (Passed)
  - t2_sizeAccumulationAcrossMultipleNestingLevels (Passed)
  - t3_searchCombinesFilterConditions (Passed)
  - t4_searchProducesDeterministicRootRelativePaths (Passed)
  - t5_statisticsAreCorrectForNestingCountingAndDepth (Passed)
  - t6_emptyStructuresAndNoMatchSearchHandled (Passed)
  - t7_repeatExecutionsDoNotLeakOrMultiplyState (Passed)
  - t8_invalidTreeMutationsRejectedWhileValidNestingSupported (Passed)

6. Traversal and Extension Trade-offs

- **Traversal Trade-off (R7):** Delegating traversal execution between DirectoryNode and visitors balances structural encapsulation with visitor control, ensuring each reachable node is visited exactly once.
- **Extension Boundary & Trade-off (R8):** Adding the third visitor (StatisticsVisitor) required zero modifications to FileNode, DirectoryNode, or their accept methods, fulfilling the open-closed principle. However, the classical trade-off of the Visitor pattern is the **Expression Problem**: adding a *new element type* (e.g., SymlinkNode) would require modifying every existing visitor interface and implementation.

7. GitHub Repository Link

Repository URL: https://github.com/Sanzh-123/assignment-6-visitor-pattern